

Why Methods Must Be Tested in Isolation?

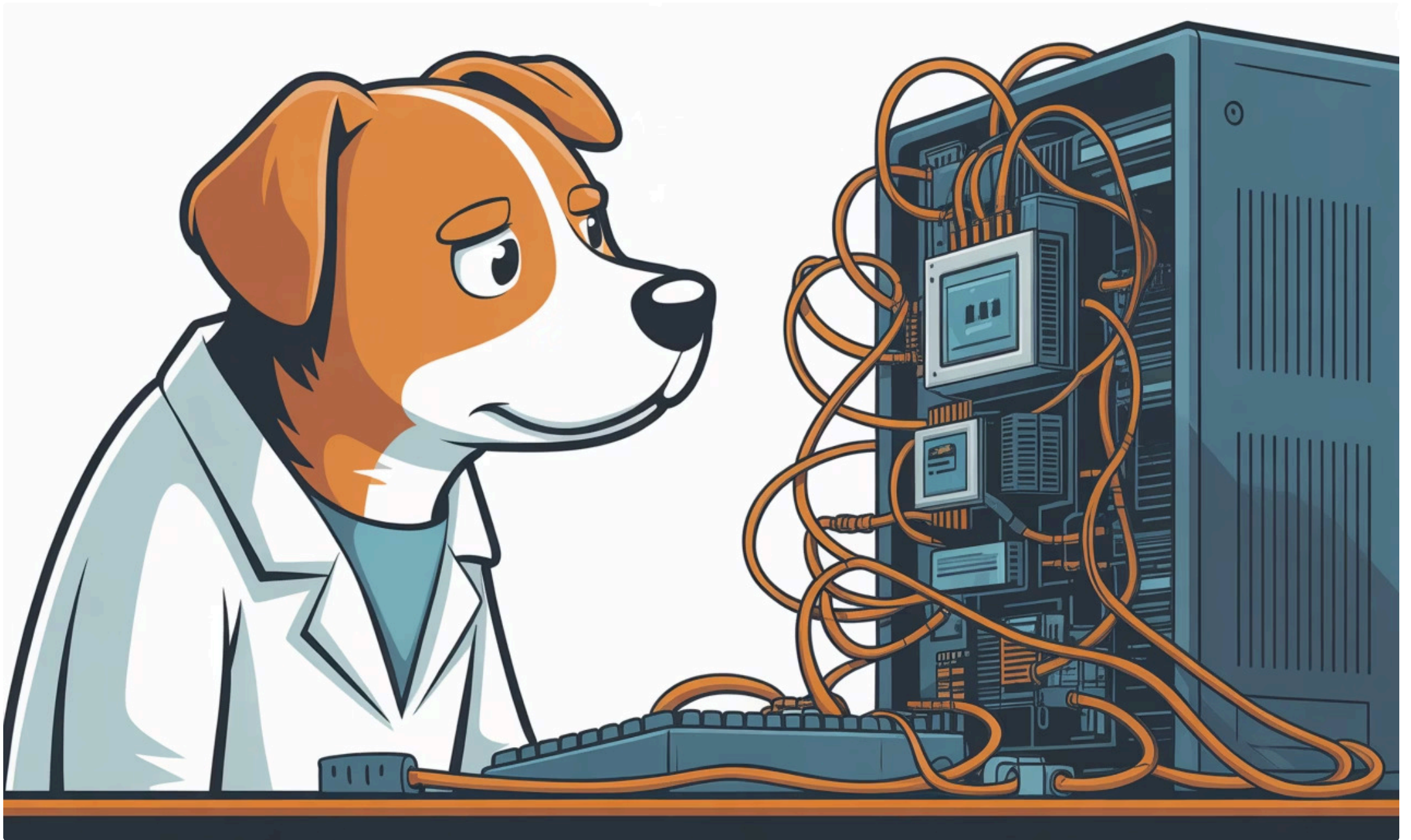
Understanding the fundamentals of unit testing and dependency management

 **by Rodrigo Martins Pagliares**



The Problem

Why Methods Must Be Tested in Isolation



Only the Logic

Unit tests should validate **only the logic** inside the method under test

Misleading Failures

External dependencies can cause misleading failures if not isolated

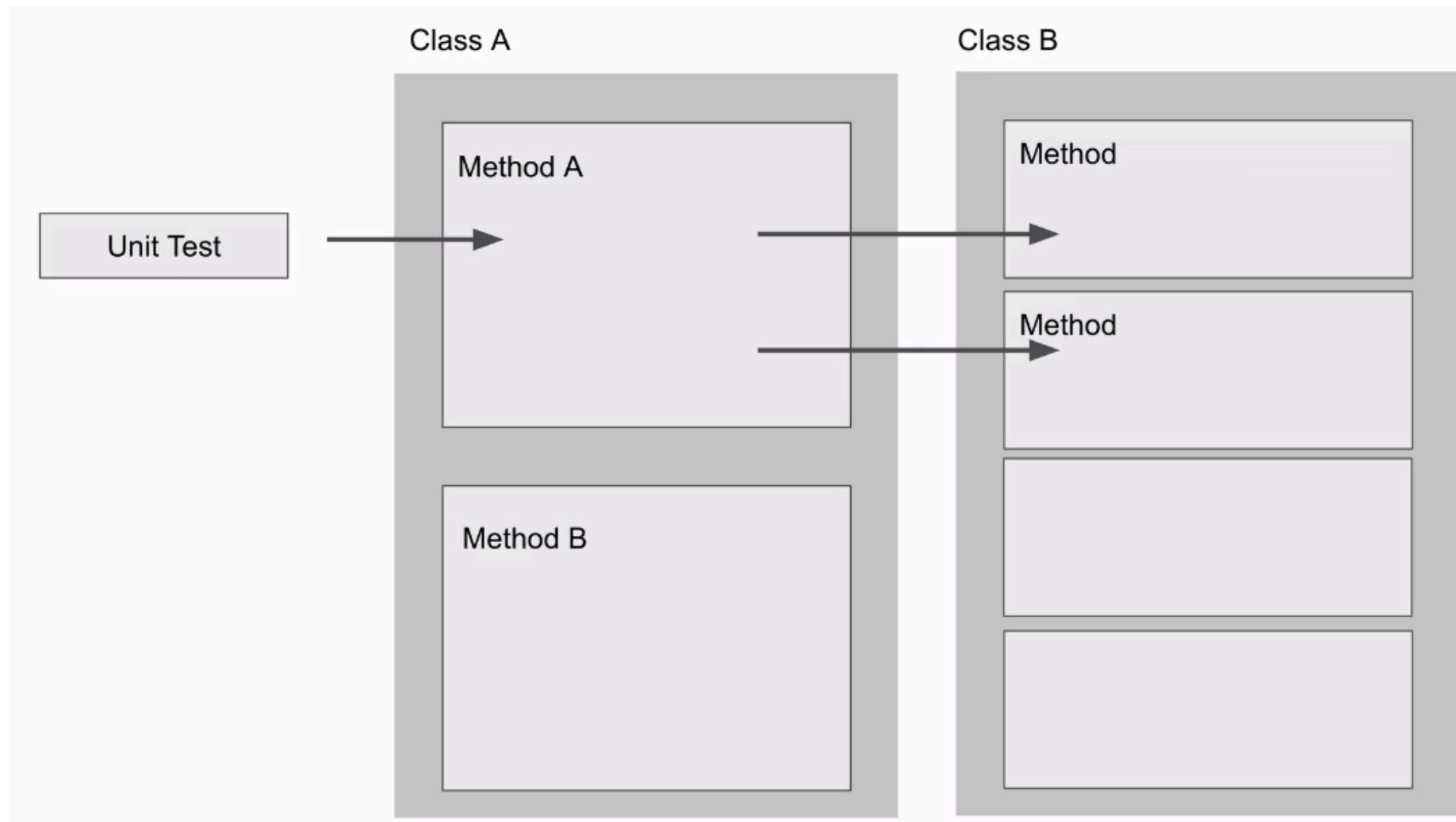
Avoid Confusion

If another object fails, the test for our method may incorrectly appear broken

Isolation ensures that only the behavior of method A is evaluated

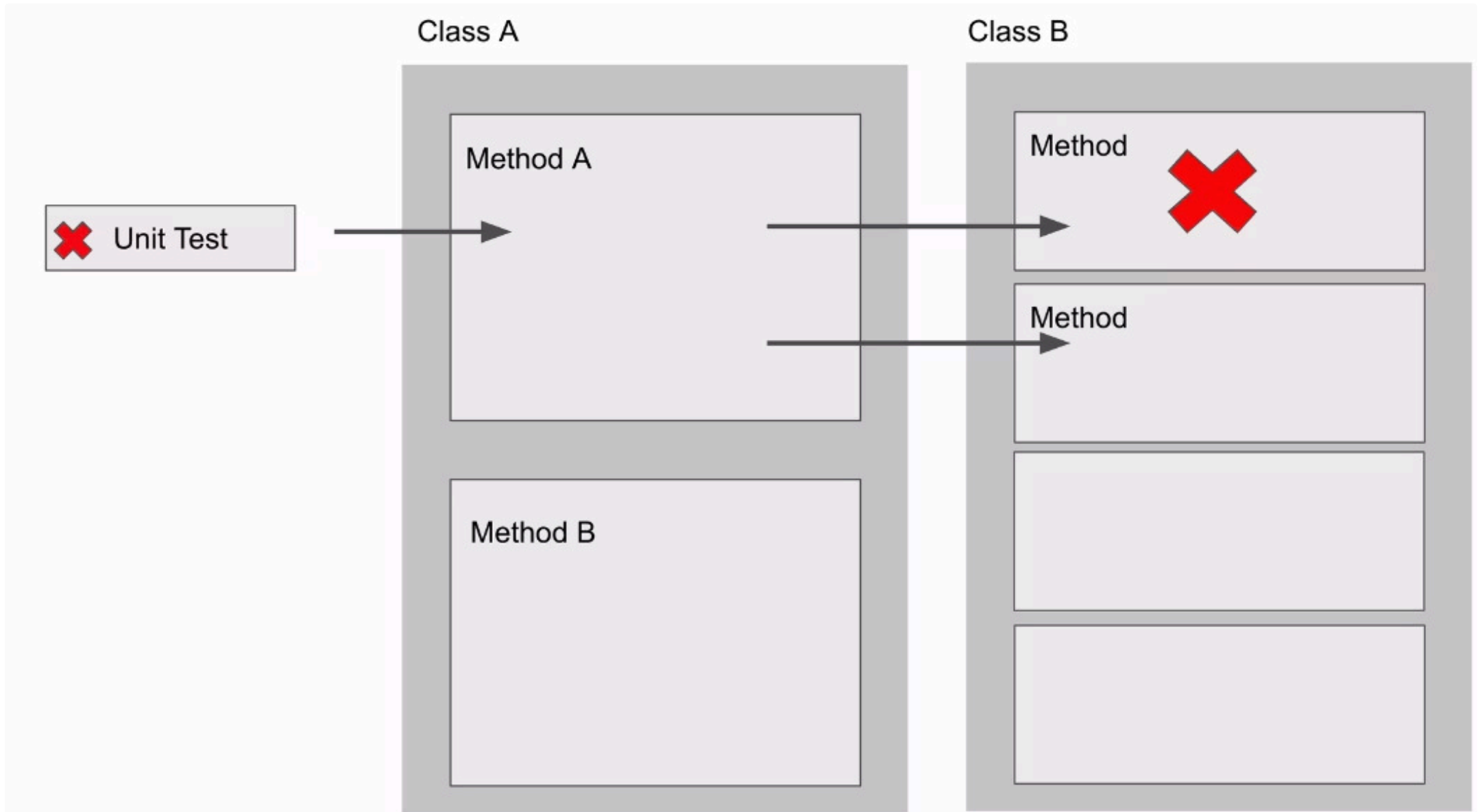
This avoids confusion about which component contains the actual defect

Why Methods Must Be Tested in Isolation?



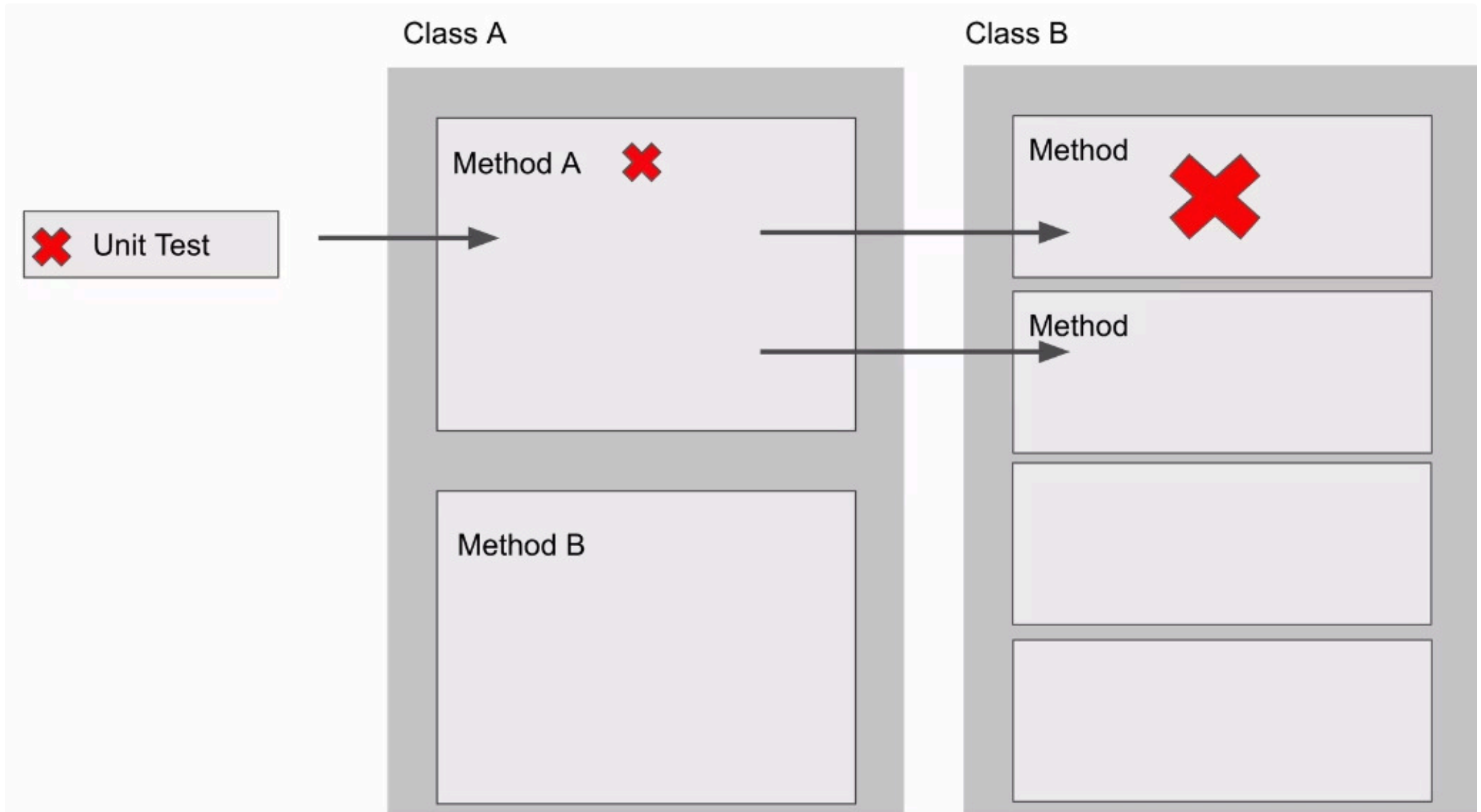
Source: Testing Java: JUnit 5, Mockito, Testcontainers, REST Assured by Sergey Kargopolov.

Why Methods Must Be Tested in Isolation?



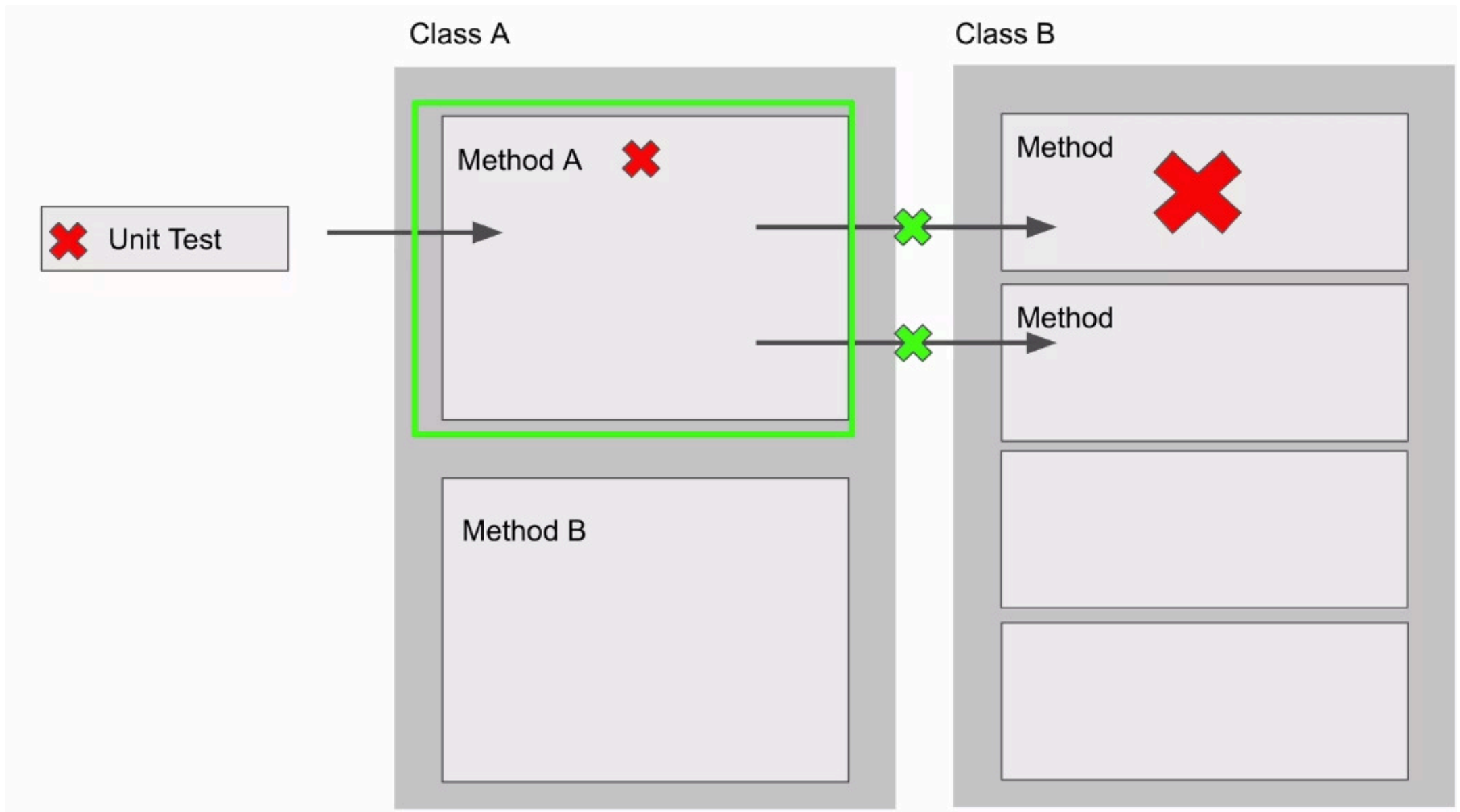
Source: Testing Java: JUnit 5, Mockito, Testcontainers, REST Assured by Sergey Kargopolov.

Why Methods Must Be Tested in Isolation?



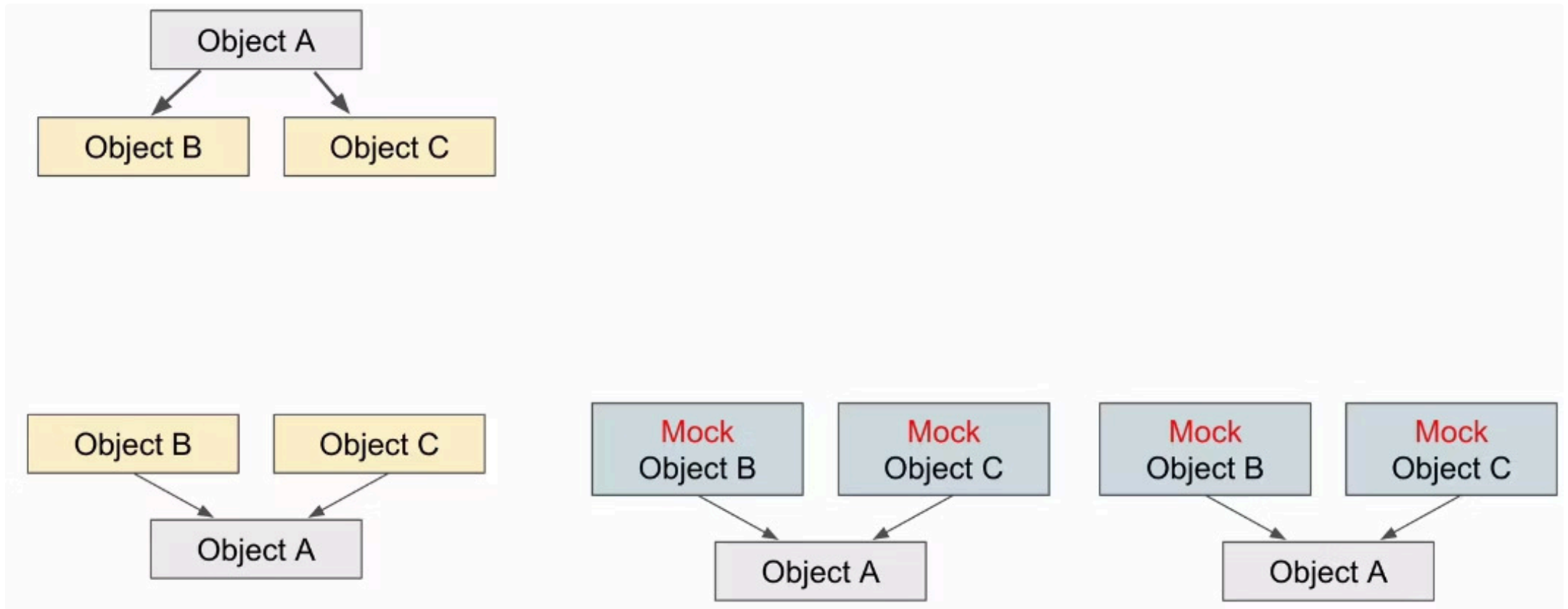
Source: Testing Java: JUnit 5, Mockito, Testcontainers, REST Assured by Sergey Kargopolov.

Why Methods Must Be Tested in Isolation?



Source: Testing Java: JUnit 5, Mockito, Testcontainers, REST Assured by Sergey Kargopolov.

Dependency Injection



Source: Testing Java: JUnit 5, Mockito, Testcontainers, REST Assured by Sergey Kargopolov.

Dependency Injection (contd.)

```
public SignupResult processUserSignup(FormData formData) {  
    SignupWebService webservice = new SignupWebServiceImpl();
```



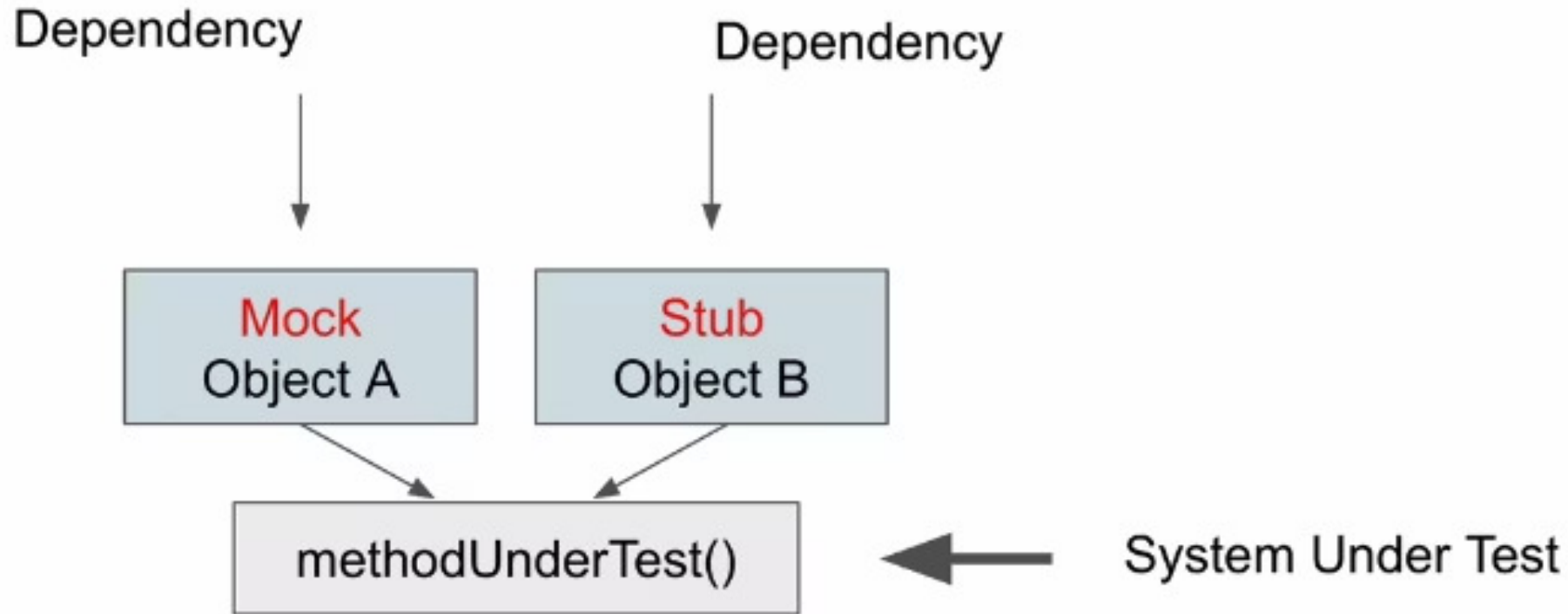
```
    return webservice.performSignup(formData);  
}
```



```
public SignupResult processUserSignup(FormData formData, SignupWebService signupWebService){  
    return webservice.performSignup(formData);  
}
```

Source: Testing Java: JUnit 5, Mockito, Testcontainers, REST Assured by Sergey Kargopolov.

Dependency Injection (contd.)



Source: Testing Java: JUnit 5, Mockito, Testcontainers, REST Assured by Sergey Kargopolov.

The Solution

Dependency Injection for Better Testability



When a method creates its own dependencies, isolation becomes difficult

Dependency Injection (DI) reverses this relationship



Provide Dependencies

Instead of method A creating B and C, these dependencies are *provided* to A



Replace with Mocks

Injected dependencies can then be replaced with mocks or stubs during testing



Test Independently

DI allows testing object A independently from B and C

Constructor injection, method parameters, or field injection can be used



Using Mocks and Stubs to **Isolate Behavior**

Mocks or stubs simulate the behavior of real dependencies without running real logic

Prevent External Failures

Prevent external failures from affecting the test

Predictable Responses

Enable **predictable**, predefined **responses**

- **Mocking** makes it easier to **test methods** with **many input variations**
- Only the **internal logic** of the **method under test** should be **evaluated**
- This **approach** increases **test reliability** and **simplifies debugging**

Wrapping Up

Conclusion

01

Isolation is Essential

Isolation is essential for accurate and meaningful unit tests

02

DI Enables Mocking

Dependency Injection enables replacing real dependencies with mocks and stubs

03

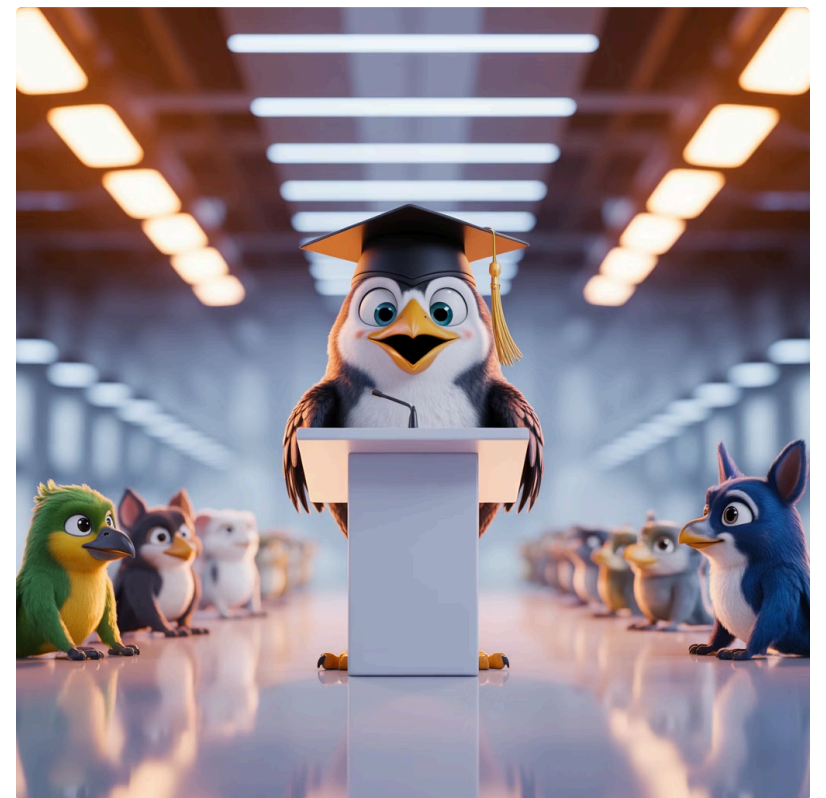
Focused Testing

Testing becomes focused, predictable, and independent of external failures

04

Increased Reliability

Well-isolated tests increase reliability and confidence in application behavior



Open Floor Discussion

- Do you typically use constructor injection or method injection in your projects?
- Which dependencies have you found hardest to mock in real applications?
- How do you decide when to use a stub vs. a mock in unit testing?

